FINDS: The Fish Interaction and Dynamics Simulator

1.0

Generated by Doxygen 1.9.8

1 FINDS: The Fish INteraction and Dynamics Simulator

FINDS was written to solve the problem of the unscalability of fish-like particle simulations used to understand the mechanics of fish swimming in schools, birds flying in groups, drones/bugs flying in swarms, and so on.

This fundamentally follows the N-body problem, as each fish-like particle (which I have equivalently called "fish automatons," "autofish," "fish- particles," and even just "particles" in various places) affects each other fish-like particle in the simulation. At a group size of N=3, an analytical solution is no longer possible, and numerical calculation is needed. As the group size N increases, the computational complexity in performing these numerical calculations grows as a function of N^2.

As such, the fundamental goal of 'Fish' was to simplify the calculations to reduce the effect of this bottleneck on Floryan's research through the implementation of approximation algorithms and speed tricks commonly used in Particle dynamics, starting with the Barnes-Hut approximation, which decreases the computational complexity to O(N log N).

Long term, the goal for the project is to try and reach computational parity with modern particle dynamics codes used in Astrophysics (and the like), such as REBOUND, GADGET-4, and similar applied to fish simulation. This is, of course, an ambitious goal for a one-man project, but we'll see how this fares.

1.1 SETUP

All that is required to install FInDS is Docker and GNU Make. For Windows users, I highly recommend using WSL and following the Linux instructions to keep your life simple, but this program can be run purely on windows regardless due to containerization.

1.1.1 Setup Example for Debian-Derived Systems

1. Download prerequisites (docker and make)

```
sudo apt update
sudo apt install git make
sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-doc \
    podman-docker containerd runc | cut -f1)

# Add Docker's official GPG key:
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o \
    /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources «EOF
Types: deb
URIs: https://download.docker.com/linux/debian
```

```
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin \
    docker-compose-plugin
```

1. Clone the Repository

```
git clone --recurse-submodules https://github.com/mufaro3/finds.git
```

1. `cd` into the repository and build the container and the documentation

```
cd finds
make build
make docs
```

1. FINDS is now ready for use. Take a look at `make help` for options on what you can do further from there.

1.1.2 Docker

Docker - Linux

All that is needed is the Docker CLI. Follow the instructions on the Docker Website.

Docker - Windows and MacOS

You probably will need Docker Desktop. Follow the instructions on the Docker Website.

1.1.3 GNU Make

GNU Make - Linux and MacOS

Your system likely comes with Make preinstalled, but if not, follow the instructions provided by the GNU foundation (ideally, download it via your package manager) at GNU.

1.2 Acknowledgements

Thanks to Mohamed Niged Mabrouk, Professor Daniel Floryan, and Alvin Ng for their guidance on this project. In particular, Alvin Ng's `grav_sim` was used extensively as a reference work for developing FINDS due to its similarity in implementation.

2 Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

3 Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

4 File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

5 Class Documentation

5.1 processingmodules.AnimationGenerator.AnimationGenerator Class Reference

Public Member Functions

- **__init__** (self, *int fps=30, int dpi=300, str video_output_filename='animation3d.mp4', ArrayLike max_bounds=[100, 100, 100], int particle_radius=30, int orientation_width=5, int orientation_length=5, float padding=0.1, tuple[int, int] figsize=(8, 8), str particle_↩ color='tab:cyan', str orientation_color='tab:orange')
- None begin (self, Path output_dir, Optional[bool] use_pdf)
- None append_state (self, dict system, float time, int frame, int num_frames)
- None end (self)

Public Attributes

- **fps**
- **dpi**
- **video_output_filename**
- **max_bounds**
- **particle_radius**
- **orientation_width**
- **orientation_length**
- **padding**
- **figsize**
- **particle_color**

- **orientation_color**
- **fig**
- **ax**
- **scatter_plot**
- **quiver_plot**
- **title_text**
- **output_filename**
- **writer**

Public Attributes inherited from processingmodules.ProcessingModule.ProcessingModule

- **output_dir**

5.1.1 Detailed Description

```
Generates a 3-D animation of the full fish simulation, taking (0,0) to be
the origin. Allows for rapid customization of simulation colors, viewing,
and debug parameters as well.

Attributes
==========
fps: int
  The frames-per-second of the simulation, or how fast the animation will
  be. Default is 30 FPS.

dpi: int
  The resolution of the video in dots per inch. Defaults to 300.

video_output_filename: str
  The output filename of the video to generate. Default is
  :code:`animation3d.mp4`

max_bounds: ArrayLike
  The maximum (unsigned) bounds for the perspective. Putting in
  :math:`(x_0, y_0, z_0)` means that the :math:`x`-axis varies from
  :math:`-x_0` to :math:`x_0`, and so on. Default is
  :math:`(100, 100, 100)`.

particle_radius: int
  The radius for each particle (default 30).

orientation_width: int
  The width for the line depicting the orientation (default 5).

orientation_length: int
  The length for the line depicting the orientation (default 5).

padding: float
  The padding for the viewport/unit box (default 0.1).
```

```
figsize: tuple[int,int]
  The size of the resulting figure (not the viewport, default
  :math:`(8,8)`).

particle_color: str
  The color of the particle, default :code:`tab:cyan`.

orientation_color: str
  The color of the orientation line, default :code:`tab:orange`.
```

5.1.2 Member Function Documentation

append_state()

```
None processingmodules.AnimationGenerator.AnimationGenerator.append_state (
            self,
           dict system,
           float time,
           int frame,
           int num_frames )


    Draws the current state to the animation.
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

begin()

```
None processingmodules.AnimationGenerator.AnimationGenerator.begin (
            self,
           Path output_dir,
           Optional[bool] use_pdf )


    Sets up the MatPlotLib animation for rendering.

    :type max_bounds: ArrayLike
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

end()

```
None processingmodules.AnimationGenerator.AnimationGenerator.end (
            self )
```

```
    Closes the animation and saves the file.
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

The documentation for this class was generated from the following file:

- scripts/processingmodules/AnimationGenerator.py

5.2 benchmark_options_t Struct Reference

Public Attributes

- size_t **size**
- derivative_computation_opts_t **dc_opts**

The documentation for this struct was generated from the following file:

- programs/benchmark_derivative.c

5.3 cmdline_args_t Struct Reference

Public Attributes

- char * **config_filepath**

The documentation for this struct was generated from the following file:

- programs/simulate.c

5.4 constant_options_t Struct Reference

Constant generation options struct.

`#include <system.h>`

Public Attributes

- bool **random_length_selection**
- float **min_length**
- float **max_length**
- float **uniform_length**
- bool **random_volumetric_flow_selection**
- float **min_sigma**
- float **max_sigma**
- float **uniform_sigma**

5.4.1 Detailed Description

Constant generation options struct.

The documentation for this struct was generated from the following file:

- include/finds/system.h

5.5 processingmodules.CrossSectionGenerator.CrossSectionGenerator Class Reference

Public Member Functions

- **__init__** (self, ∗float z_thickness=0.25, float particle_radius=30, float orientation_↩ width=0.25, float orientation_length=1, tuple[int, int] figsize=(10, 10), str particle_↩ color='tab:cyan', str orientation_color='tab:orange', str output_filename="cross_section", int dpi=200)
- begin (self, Path output_dir, Optional[bool] use_pdf)
- append_state (self, NDArray system, float time, int frame, int num_frames)
- end (self)

Public Attributes

- **z_thickness**
- **particle_radius**
- **orientation_width**
- **orientation_length**
- **figsize**
- **particle_color**
- **orientation_color**
- **output_filename**
- **dpi**
- **output_dir**
- **generated**

Public Attributes inherited from processingmodules.ProcessingModule.ProcessingModule

- **output_dir**

5.5.1 Detailed Description

```
Generates a cross-section for the first frame along the xy-plane at z=0.

Attributes
==========
particle_radius: int
  The radius for each particle (default 30).

orientation_width: float
  The width for the line depicting the orientation (default 0.25).

orientation_length: float
  The length for the line depicting the orientation (default 5).

figsize: tuple[int,int]
  The size of the resulting figure (not the viewport, default
  :math:`(10,10)`).

particle_color: str
  The color of the particle, default :code:`tab:cyan`.

orientation_color: str
  The color of the orientation line, default :code:`tab:orange`.

output_filename: str
  The output filename of the image to generate. Default is
  :code:`cross_section`

generated: bool
  Whether or not we have already generated the cross-section figure.

dpi: int
  The resolution of the fig in dots per inch. Defaults to 200.
```

5.5.2 Member Function Documentation

append_state()

```
processingmodules.CrossSectionGenerator.CrossSectionGenerator.append_state
(
            self,
           NDArray system,
           float time,
           int frame,
           int num_frames )
```

```
    Appends the state :math:`\mathbf{X}` and time :math:`t` to the current
    process being generated (whether by animating it, including it in a
    computation, etc.).

    :param system: The state of the system at time :math:`t`.
    :type  system: NDArray

    :param time: The time, :math:`t`.
    :type  time: float

    :param frame: The index for this frame
    :type  frame: int

    :param num_frames: The total number of frames for this simulation
    :type  num_frames: int
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

begin()

```
processingmodules.CrossSectionGenerator.CrossSectionGenerator.begin (
            self,
           Path output_dir,
           Optional[bool] use_pdf )
```

```
Instructs the processing module to initialize such that it can
produce data.

:param output_dir: The output directory to write to.
:type  output_dir: Path

:param use_pdf: Whether or not to output to SVG.
:type  use_pdf: bool
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

end()

```
processingmodules.CrossSectionGenerator.CrossSectionGenerator.end (
            self )
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

The documentation for this class was generated from the following file:

- scripts/processingmodules/CrossSectionGenerator.py

5.6 datastream_t Struct Reference

File I/O structure for HDF5 output.

```
#include <datastream.h>
```

Public Attributes

- bool **open**
- hid_t **file**
- hid_t **time_dataset**
- hid_t **position_dataset**
- hid_t **orientation_dataset**
- hid_t **length_dataset**
- hid_t **sigma_dataset**
- size_t **n_particles**
- size_t **n_frames**

5.6.1 Detailed Description

File I/O structure for HDF5 output.

The documentation for this struct was generated from the following file:

- include/finds/datastream.h

5.7 processingmodules.DensityAnimationGenerator.DensityAnimation↩ Generator Class Reference

Public Member Functions

- **__init__** (self, ∗int fps=30, int dpi=300, int n_bins=30, float max_radius=100.0, tuple[int, int] figsize=(6, 4), str video_output_filename='density_animation.mp4')
- None begin (self, Path output_dir, Optional[bool] use_pdf)
- None append_state (self, NDArray system, float time, int frame, int num_frames)
- None end (self)

Public Attributes

- **fps**
- **dpi**
- **n_bins**
- **max_radius**
- **figsize**
- **video_output_filename**
- **output_dir**
- **fig**
- **ax**
- **bins**
- **bars**
- **title_text**
- **output_filename**
- **writer**

Public Attributes inherited from processingmodules.ProcessingModule.ProcessingModule

- **output_dir**

5.7.1 Detailed Description

```
Produces an animation of the radial mass distribution of fish from the
center-of-mass for each time step, :math:`\delta t`.

Attributes
==========
fps: int
  The frames-per-second of the animation. Default 30 FPS.

dpi: int
  The dots-per-inch of the animation. Default 300.

n_bins: int
  The number of bins to use in the histogram. Default 30.

max_radius: float
  The maximum radius to display on the histogram. Default 100.

figsize: tuple[int,int]
  The MatPlotLib figure dimensions for the video. Default (6,4).

video_output_filename: str
  The filename to output the video to.
```

5.7.2 Member Function Documentation

append_state()

```
None processingmodules.DensityAnimationGenerator.DensityAnimationGenerator.↩
append_state (
            self,
           NDArray system,
           float time,
           int frame,
           int num_frames )

    Appends the state :math:`\mathbf{X}` and time :math:`t` to the current
    process being generated (whether by animating it, including it in a
    computation, etc.).

    :param system: The state of the system at time :math:`t`.
    :type  system: NDArray

    :param time: The time, :math:`t`.
    :type  time: float

    :param frame: The index for this frame
    :type  frame: int

    :param num_frames: The total number of frames for this simulation
    :type  num_frames: int
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

begin()

```
None processingmodules.DensityAnimationGenerator.DensityAnimationGenerator.↩
begin (
            self,
           Path output_dir,
           Optional[bool] use_pdf )


Instructs the processing module to initialize such that it can
produce data.

:param output_dir: The output directory to write to.
:type  output_dir: Path

:param use_pdf: Whether or not to output to SVG.
:type  use_pdf: bool
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

end()

```
None processingmodules.DensityAnimationGenerator.DensityAnimationGenerator.↩
end (
            self )
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

The documentation for this class was generated from the following file:

- scripts/processingmodules/DensityAnimationGenerator.py

5.8 derivative_computation_opts_t Struct Reference

Derivative computation options struct.

```
#include <derivative.h>
```

Public Attributes

- interaction_computation_methods_e **method**
- double **approximation_threshold**
- double **precision**
- double **regularization_epsilon**
- bool **regularize**

5.8.1 Detailed Description

Derivative computation options struct.

The documentation for this struct was generated from the following file:

- include/finds/derivative.h

5.9 derivative_weight_t Struct Reference

Weight structure for computing weighted averages of derivatives.

`#include <derivative.h>`

Public Attributes

- double **coeff**
- system_derivative_t * **deriv**

5.9.1 Detailed Description

Weight structure for computing weighted averages of derivatives.

The documentation for this struct was generated from the following file:

- include/finds/derivative.h

5.10 distribution_options_t Struct Reference

Distribution generation options struct.

`#include <system.h>`

Public Attributes

- distribution_type_e **type**
- union {
  double **radius**
  double **side_length**
  double **size_random**
  };

- union {
  double **spacing**
  double **abs_bound**
  };

5.10.1 Detailed Description

Distribution generation options struct.

The documentation for this struct was generated from the following file:

- include/finds/system.h

5.11 double_3d_t Union Reference

3-dimensional cartesian vector implementation

```
#include <vector.h>
```

Public Attributes

- struct {
  double **x**
  double **y**
  double **z**
  };

- double **data** [3]

5.11.1 Detailed Description

3-dimensional cartesian vector implementation

The documentation for this union was generated from the following file:

- include/finds/vector.h

5.12 feature_positions_t Struct Reference

Vector of feature positions.

`#include <system.h>`

Public Attributes

- swimmer_features_t * **swimmers**
- size_t **size**

5.12.1 Detailed Description

Vector of feature positions.

The documentation for this struct was generated from the following file:

- include/finds/system.h

5.13 feature_velocity_t Struct Reference

Local alias for the derivative of feature_positions_t for clarity.

Public Attributes

- double_3d_t * **source**
- double_3d_t * **sink**
- size_t **size**

5.13.1 Detailed Description

Local alias for the derivative of feature_positions_t for clarity.

The documentation for this struct was generated from the following file:

- src/derivative.c

5.14 finds_opts_t Struct Reference

Public Attributes

- distribution_options_t **dist_opts**
- orientation_options_t **ori_opts**
- constant_options_t **const_opts**
- derivative_computation_opts_t **dc_opts**
- integration_opts_t **int_opts**
- bool **print_output_file**
- bool **print_generated_system**

5.14.1 Detailed Description

Simulation options for use in loading the TOML file.

The documentation for this struct was generated from the following file:

- programs/simulate.c

5.15 fish_system_t Struct Reference

Struct for a system of swimmers (i.e., a vector of swimmers).

```
#include <system.h>
```

Public Attributes

- swimmer_t * **swimmers**
- size_t **size**

5.15.1 Detailed Description

Struct for a system of swimmers (i.e., a vector of swimmers).

The documentation for this struct was generated from the following file:

- include/finds/system.h

5.16 integration_opts_t Struct Reference

Struct for integration options.

`#include <integration.h>`

Public Attributes

- integration_method_e **method**
- double **eval_time_step**
- double **end_time**
- double **relative_error_tolerance**
- double **absolute_error_tolerance**
- bool **print_time_progression**

5.16.1 Detailed Description

Struct for integration options.

The documentation for this struct was generated from the following file:

- include/finds/integration.h

5.17 linear_octree_builder_t Struct Reference

A variable-sized wrapper for building linear octrees.

Public Attributes

- size_t **capacity**
- linear_octree_t **tree**

5.17.1 Detailed Description

A variable-sized wrapper for building linear octrees.

Growable builder that is trimmed to exact size once construction finishes. Defined to be variable-size so that the actual linear octree can be nicely fixed-size upon construction. Otherwise, it is completely identical to the linear octree.

The documentation for this struct was generated from the following file:

- src/barneshut.c

5.18 linear_octree_t Struct Reference

Linear octree implementation for swimmer data.

```
#include <barneshut.h>
```

Public Attributes

- uint64_t **count**
- double_3d_t * **centers**
- double * **side_lengths**
- child_indices_t * **child_indices**
- bool * **is_leaf**
- size_t * **swimmer_index**
- size_t * **num_particles**
- double_3d_t * **weighted_positions_sums**
- double_3d_t * **weighted_orientations_sums**
- double * **weighted_length_sums**
- double * **volumetric_flow_rate_sums**
- double_3d_t * **source_position_avg**
- double_3d_t * **sink_position_avg**

5.18.1 Detailed Description

Linear octree implementation for swimmer data.

The documentation for this struct was generated from the following file:

- include/finds/barneshut.h

5.19 mat3_t Struct Reference

3x3 matrix implementation

```
#include <vector.h>
```

Public Attributes

- double **data** [3][3]

5.19.1 Detailed Description

3x3 matrix implementation

The documentation for this struct was generated from the following file:

- include/finds/vector.h

5.20 processingmodules.MeanRadialDistancePlot.MeanRadialDistance↩ Plot Class Reference

Public Member Functions

- **__init__** (self, ∗int dpi=300, tuple[int, int] figsize=(10, 10), str output_filename='mean_↩ radial_distance')
- None begin (self, Path output_dir, Optional[bool] use_pdf)
- None append_state (self, NDArray system, float time, int frame, int num_frames)
- None end (self)

Public Attributes

- **dpi**
- **figsize**
- **output_filename**
- **output_dir**
- **output_filepath**
- **std_radial_distances**

Public Attributes inherited from processingmodules.ProcessingModule.ProcessingModule

- **output_dir**

5.20.1 Detailed Description

```
Produces a plot of the mean radial mass distribution of fish from the
center-of-mass as a function of time, including errorbars for the standard
deviations.

Attributes
==========
dpi: int
  The dots-per-inch of the animation. Default 300.

figsize: tuple[int,int]
  The MatPlotLib figure dimensions for the video. Default (10,10).

output_filename: str
  The filename to output the figure to.
```

5.20.2 Member Function Documentation

append_state()

```
None processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot.append↩
_state (
            self,
           NDArray system,
           float time,
           int frame,
           int num_frames )
```

```
    Appends the state :math:`\mathbf{X}` and time :math:`t` to the current
    process being generated (whether by animating it, including it in a
    computation, etc.).

    :param system: The state of the system at time :math:`t`.
    :type  system: NDArray

    :param time: The time, :math:`t`.
    :type  time: float

    :param frame: The index for this frame
    :type  frame: int

    :param num_frames: The total number of frames for this simulation
    :type  num_frames: int
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

begin()

```
None processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot.begin
(
            self,
           Path output_dir,
           Optional[bool] use_pdf )


Instructs the processing module to initialize such that it can
produce data.

:param output_dir: The output directory to write to.
:type  output_dir: Path

:param use_pdf: Whether or not to output to SVG.
:type  use_pdf: bool
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

end()

```
None processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot.end (
            self )
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

The documentation for this class was generated from the following file:

- scripts/processingmodules/MeanRadialDistancePlot.py

5.21 morton_key_t Struct Reference

This maps morton codes to node indices in a linear octree.

Public Attributes

- uint64_t **morton_code**
- size_t **assoc_node_index**

5.21.1 Detailed Description

This maps morton codes to node indices in a linear octree.

The documentation for this struct was generated from the following file:

- src/barneshut.c

5.22 named_enum_t Struct Reference

Enum to name string mapping struct.

```
#include <util.h>
```

Public Attributes

- int **enum_value**
- char * **name**

5.22.1 Detailed Description

Enum to name string mapping struct.

The documentation for this struct was generated from the following file:

- include/finds/util.h

5.23 orientation_options_t Struct Reference

Orientation generation options struct.

`#include <system.h>`

Public Attributes

- orientation_type_e **type**
- float **angular_perturbation**

5.23.1 Detailed Description

Orientation generation options struct.

The documentation for this struct was generated from the following file:

- include/finds/system.h

5.24 processingmodules.ProcessingModule.ProcessingModule Class Reference

Public Member Functions

- begin (self, Path output_dir, Optional[bool] use_pdf)
- None append_state (self, NDArray system, float time, int frame, int num_frames)
- None end (self)

Public Attributes

- **output_dir**

5.24.1 Detailed Description

```
This is an abstract class for all "processing modules" so that they can
operate in parallel to reduce the amount of data read to the disk at one
time (such as generating figures, performing calculations, etc).
```

5.24.2 Member Function Documentation

append_state()

```
None processingmodules.ProcessingModule.ProcessingModule.append_state (
            self,
           NDArray system,
           float time,
           int frame,
           int num_frames )

    Appends the state :math:`\mathbf{X}` and time :math:`t` to the current
    process being generated (whether by animating it, including it in a
    computation, etc.).

    :param system: The state of the system at time :math:`t`.
    :type  system: NDArray

    :param time: The time, :math:`t`.
    :type  time: float

    :param frame: The index for this frame
    :type  frame: int

    :param num_frames: The total number of frames for this simulation
    :type  num_frames: int
```

Reimplemented in processingmodules.AnimationGenerator.AnimationGenerator, processingmodules.CrossSec
processingmodules.DensityAnimationGenerator.DensityAnimationGenerator, processingmodules.MeanRadialD
processingmodules.TrajectoryPlot.TrajectoryPlot, and processingmodules.SnapshotGenerator.SnapshotGenera

begin()

```
processingmodules.ProcessingModule.ProcessingModule.begin (
            self,
           Path output_dir,
           Optional[bool] use_pdf )

Instructs the processing module to initialize such that it can
produce data.

:param output_dir: The output directory to write to.
:type  output_dir: Path

:param use_pdf: Whether or not to output to SVG.
:type  use_pdf: bool
```

Reimplemented in processingmodules.AnimationGenerator.AnimationGenerator, processingmodules.CrossSec
processingmodules.DensityAnimationGenerator.DensityAnimationGenerator, processingmodules.MeanRadialD
processingmodules.SnapshotGenerator.SnapshotGenerator, and processingmodules.TrajectoryPlot.TrajectoryP

end()

```
None processingmodules.ProcessingModule.ProcessingModule.end (
            self )
```

Reimplemented in processingmodules.AnimationGenerator.AnimationGenerator.

The documentation for this class was generated from the following file:

- scripts/processingmodules/ProcessingModule.py

5.25 processingmodules.SnapshotGenerator.SnapshotGenerator Class Reference

Public Member Functions

- **__init__** (self, *str output_filename="snapshot", str output_filename_ending="png", ArrayLike max_bounds=[100, 100, 100], int particle_radius=30, float orientation_↩ length=5, float padding=0.1, tuple[int, int] figsize=(8, 8), str particle_color="tab:cyan", str orientation_color="tab:orange")
- None begin (self, Path output_dir, Optional[bool] use_pdf)
- None append_state (self, system, float time, int frame, int num_frames)
- None end (self)

Public Attributes

- **output_filename**
- **output_filename_ending**
- **max_bounds**
- **particle_radius**
- **orientation_length**
- **padding**
- **figsize**
- **particle_color**
- **orientation_color**
- **output_dir**
- **first**
- **middle**
- **last**

Public Attributes inherited from processingmodules.ProcessingModule.ProcessingModule

- **output_dir**

Protected Member Functions

- **_draw_snapshot** (self, positions, orientations, time, index)

5.25.1 Detailed Description

```
Generates a three-panel snapshot of the simulation showing the first,
middle, and last simulation frames.
```

5.25.2 Member Function Documentation

append_state()

```
None processingmodules.SnapshotGenerator.SnapshotGenerator.append_state (
            self,
            system,
           float time,
           int frame,
           int num_frames )
```

```
    Appends the state :math:`\mathbf{X}` and time :math:`t` to the current
    process being generated (whether by animating it, including it in a
    computation, etc.).

    :param system: The state of the system at time :math:`t`.
    :type  system: NDArray

    :param time: The time, :math:`t`.
    :type  time: float

    :param frame: The index for this frame
    :type  frame: int

    :param num_frames: The total number of frames for this simulation
    :type  num_frames: int
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

begin()

```
None processingmodules.SnapshotGenerator.SnapshotGenerator.begin (
            self,
           Path output_dir,
           Optional[bool] use_pdf )


Instructs the processing module to initialize such that it can
produce data.

:param output_dir: The output directory to write to.
:type  output_dir: Path

:param use_pdf: Whether or not to output to SVG.
:type  use_pdf: bool
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

end()

```
None processingmodules.SnapshotGenerator.SnapshotGenerator.end (
            self )
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

The documentation for this class was generated from the following file:

- scripts/processingmodules/SnapshotGenerator.py

5.26 swimmer_features_t Struct Reference

Structure of feature (source and sink) positions.

```
#include <system.h>
```

Public Attributes

- double_3d_t **source**
- double_3d_t **sink**

5.26.1 Detailed Description

Structure of feature (source and sink) positions.

The documentation for this struct was generated from the following file:

- include/finds/system.h

5.27 swimmer_t Struct Reference

Structure representing an individual swimmer.

`#include <system.h>`

Public Attributes

- double_3d_t **position**
- double_3d_t **orientation**
- double **volumetric_flow_rate**
- double **length**
- double **sp_speed**

5.27.1 Detailed Description

Structure representing an individual swimmer.

The documentation for this struct was generated from the following file:

- include/finds/system.h

5.28 system_derivative_t Struct Reference

System derivative (translational and rotational derivatives).

`#include <derivative.h>`

Public Attributes

- double_3d_t * **translational**
- double_3d_t * **rotational**
- size_t **size**

5.28.1 Detailed Description

System derivative (translational and rotational derivatives).

The documentation for this struct was generated from the following file:

- include/finds/derivative.h

5.29 system_trajectory_t Struct Reference

Swimmer trajectory object for simplifying the coplanar cases.

Public Attributes

- size_t **num_frames**
- size_t **num_swimmers**
- swimmer_t ** **data**

5.29.1 Detailed Description

Swimmer trajectory object for simplifying the coplanar cases.

The documentation for this struct was generated from the following file:

- programs/validation.c

5.30 processingmodules.TrajectoryPlot.TrajectoryPlot Class Reference

Public Member Functions

- **__init__** (self, *int dpi=300, tuple[int, int] figsize=(10, 10), str output_filename='trajectory↩
_plot', int particle_radius=30)
- None begin (self, Path output_dir, Optional[bool] use_pdf)
- None append_state (self, NDArray system, float time, int frame, int num_frames)
- None end (self)

Public Attributes

- **dpi**
- **figsize**
- **output_filename**
- **particle_radius**
- **output_dir**
- **output_filepath**
- **position_frames**

Public Attributes inherited from processingmodules.ProcessingModule.ProcessingModule

- **output_dir**

5.30.1 Detailed Description

```
Produces a plot of the mean radial mass distribution of fish from the
center-of-mass as a function of time, including errorbars for the standard
deviations.

Attributes
==========
dpi: int
  The dots-per-inch of the animation. Default 300.

figsize: tuple[int,int]
  The MatPlotLib figure dimensions for the video. Default (10,10).

plot_output_filename: str
  The filename to output the figure to (default
  :code:`trajectory_plot.png`).

particle_radius: int
  The radius of each fish-particle (default 30).
```

5.30.2 Member Function Documentation

append_state()

```
None processingmodules.TrajectoryPlot.TrajectoryPlot.append_state (
            self,
           NDArray system,
           float time,
           int frame,
           int num_frames )
```

```
    Appends the state :math:`\mathbf{X}` and time :math:`t` to the current
    process being generated (whether by animating it, including it in a
    computation, etc.).

    :param system: The state of the system at time :math:`t`.
    :type  system: NDArray

    :param time: The time, :math:`t`.
    :type  time: float

    :param frame: The index for this frame
    :type  frame: int

    :param num_frames: The total number of frames for this simulation
    :type  num_frames: int
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

begin()

```
None processingmodules.TrajectoryPlot.TrajectoryPlot.begin (
            self,
           Path output_dir,
           Optional[bool] use_pdf )
```

```
Instructs the processing module to initialize such that it can
produce data.

:param output_dir: The output directory to write to.
:type  output_dir: Path

:param use_pdf: Whether or not to output to SVG.
:type  use_pdf: bool
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

end()

```
None processingmodules.TrajectoryPlot.TrajectoryPlot.end (
            self )
```

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

The documentation for this class was generated from the following file:

- scripts/processingmodules/TrajectoryPlot.py

6 File Documentation

6.1 include/finds/barneshut.h File Reference

External velocity contribution calculation through Barnes-Hut.

```
#include <stdint.h>
#include "system.h"
#include "vector.h"
#include "error.h"
```

Classes

- struct linear_octree_t

  Linear octree implementation for swimmer data.

Macros

- #define **OCTREE_NO_CHILD** UINT64_MAX

  Sentinel value for "no child in this octant".

Typedefs

- typedef uint64_t **child_indices_t**[8]

  Typedef for size-8 indices.

Functions

- bool linear_octree_check_nan (const linear_octree_t *restrict tree, bool verbose)

  Checks if a linear octree has any NaN values.
- void linear_octree_print (const linear_octree_t *restrict tree)

  Prints a full octree to STDOUT (for debugging).
- linear_octree_t * linear_octree_build (const fish_system_t *restrict system)

  Builds a linear octree from a fish system.
- void linear_octree_destroy (linear_octree_t **octree_ptr)

  De-allocates a linear octree.
- void linear_octree_compute_vel_contrib (const linear_octree_t *restrict tree, const size_t swimmer_i_index, const double_3d_t swimmer_i_pos, const swimmer_features_t features_i, const double approx_ratio, const bool regularize, const double epsilon, double_3d_t *restrict external_source, double_3d_t *restrict external_sink)

  Computes the velocity contribution on a swimmer by all of the non-self swimmers in the system.

6.1.1 Detailed Description

External velocity contribution calculation through Barnes-Hut.

This file contains an implementation of the Barnes-Hut algorithm for external velocity contribution using linear octrees as a backend.

Author

Mufaro Machaya mufaro2@student.ubc.ca

6.1.2 Function Documentation

linear_octree_build()

```
linear_octree_t * linear_octree_build (
          const fish_system_t *restrict system )
```

Builds a linear octree from a fish system.

Parameters

in	*system*	The fish system to construct from.

Returns

The constructed linear octree.

linear_octree_check_nan()

```
bool linear_octree_check_nan (
          const linear_octree_t *restrict tree,
          bool verbose )
```

Checks if a linear octree has any NaN values.

Parameters

in	*tree*	The octree.
in	*verbose*	Whether to print the exact value which contains NaN.

Returns

Whether or not any NaN values were found inside the tree.

linear_octree_compute_vel_contrib()

```
void linear_octree_compute_vel_contrib (
          const linear_octree_t *restrict tree,
          const size_t swimmer_i_index,
          const double_3d_t swimmer_i_pos,
          const swimmer_features_t features_i,
          const double approx_ratio,
          const bool regularize,
          const double eps,
          double_3d_t *restrict external_source,
          double_3d_t *restrict external_sink )
```

Computes the velocity contribution on a swimmer by all of the non-self swimmers in the system.

Parameters

in	*tree*	The linear octree.
in	*swimmer_i_index*	The swimmer index for swimmer i.
in	*swimmer_i_pos*	The position of swimmer i.
in	*features_i*	The feature positions of swimmer i.
in	*approx_ratio*	The Barnes-Hut approximation ratio.
in	*regularize*	Whether or not to use regularized interaction.
in	*eps*	Regularization Epsilon
out	*external_source*	The output vec. for external source contrib.
out	*external_sink*	The output vec. for external sink contrib.

linear_octree_destroy()

```
void linear_octree_destroy (
          linear_octree_t ** octree_ptr )
```

De-allocates a linear octree.

Parameters

out	*octree_ptr*	The point to the octree to destroy.

linear_octree_print()

```
void linear_octree_print (
          const linear_octree_t *restrict tree )
```

Prints a full octree to STDOUT (for debugging).

Parameters

in	*tree*	The octree to print.

6.2 include/finds/constants.h File Reference

Various constants for the FINDS system.

Macros

- #define **ROOT_OUTPUT_PATH** "output"
- #define **SIMULATIONS_OUTPUT_PATH** "simulations"
- #define **DATAFILE_NAME** "data.h5"
- #define **SAFETY** 0.9f
- #define **MODE_RW_USERONLY** 0755

6.2.1 Detailed Description

Various constants for the FINDS system.

Author

Mufaro Machaya

License: MIT

6.3 include/finds/datastream.h File Reference

I/O routines.

```
#include <hdf5.h>
#include <stdbool.h>
#include "system.h"
#include "error.h"
```

Classes

- struct datastream_t

  File I/O structure for HDF5 output.

Functions

- error_e mkdir_p (const char *path, const mode_t mode)

  C implementation of "mkdir -p" command.
- error_e datastream_open_file (datastream_t *stream, const char *filename)

  Opens a dataset file for reading.
- error_e **datastream_read_frame** (datastream_t *stream, const size_t frame, fish_system_t **dest_ptr, double *time)
- error_e datastream_create_file (datastream_t *stream, const char *filename, const size_t N)

  Creates the root file for the datastream and marks it as open.
- error_e datastream_close (datastream_t *stream)

  Closes this datastream.
- error_e datastream_write_system (datastream_t *stream, const fish_system_t *system, const double time)

  Writes the state of a single system out to the datastream.

6.3.1 Detailed Description

I/O routines.

This contains the implementation of the datastream object, which is used for creating and writing to simulation data files.

Author

Mufaro Machaya

License: MIT

6.3.2 Function Documentation

datastream_close()

```
error_e datastream_close (
          datastream_t * stream )
```

Closes this datastream.

Parameters

out	*stream*	The stream to close.

Returns

The error code for this process.

datastream_create_file()

```
error_e datastream_create_file (
          datastream_t * stream,
          const char * filename,
          const size_t N )
```

Creates the root file for the datastream and marks it as open.

Parameters

out	*stream*	The datastream to open.
in	*filename*	The output filename for the dataset.
in	*N*	The number of swimmers to track in this dataset.

Returns

The error code for this process.

datastream_open_file()

```
error_e datastream_open_file (
          datastream_t * stream,
          const char * filename )
```

Opens a dataset file for reading.

Parameters

out	*stream*	The datastream to read to.
in	*filename*	The file to open.

Returns

The error code.

datastream_write_system()

```
error_e datastream_write_system (
          datastream_t * stream,
          const fish_system_t * system,
          const double time )
```

Writes the state of a single system out to the datastream.

Parameters

out	*stream*	The datastream to write to.
in	*system*	The system state to write.
in	*time*	The time marker to write.

Returns

The error code for this operation.

mkdir_p()

```
error_e mkdir_p (
          const char * path,
          const mode_t mode )
```

C implementation of "mkdir -p" command.

Note: I'll be honest, I just stole this from generative AI. It works quite okay, though. Another possible implementation could just directly use mkdir -p via a system call (as this code doesn't need to be portable due to running in a virtual machine).

Parameters

in	*path*	The folder path to create.
in	*mode*	The mkdir creation mode.

Returns

The error code for this process.

6.4 include/finds/derivative.h File Reference

Derivative calculation.

```
#include <string.h>
#include <stdlib.h>
#include "vector.h"
#include "system.h"
#include "util.h"
```

Classes

- struct system_derivative_t

  System derivative (translational and rotational derivatives).
- struct derivative_computation_opts_t

  Derivative computation options struct.
- struct derivative_weight_t

  Weight structure for computing weighted averages of derivatives.

Enumerations

- enum interaction_computation_methods_e { **BRUTE_FORCE** , **BARNES_HUT** , **FAST_MULTIPOLE_METHOD** , **INTER_COMP_METHODS_COUNT** }

  Interaction computation methods list.

Functions

- system_derivative_t * compute_system_derivative (const fish_system_t *system, const derivative_computation_opts_t dc_opts)

  Computes the derivative for a fish system.
- bool derivative_has_nan (const system_derivative_t *derivative)

  Determines whether a system derivative contains a NaN or Inf value (for debugging).
- void derivative_destroy (system_derivative_t **derivative)

  De-allocates a derivative object.
- void derivative_print (const system_derivative_t *derivative)

  Prints a derivative (for debugging).
- system_derivative_t * derivative_average (const derivative_weight_t terms[ ], const size_t N, const size_t len)

  Computes the weighted average of a derivative.

Variables

- const named_enum_t **INTER_COMP_METHODS_TABLE** [INTER_COMP_METHODS_COUNT]

  Lookup table for the interaction computation methods.

6.4.1 Detailed Description

Derivative calculation.

This file contains the core routine for computing the derivative of the fish system.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.4.2 Function Documentation

compute_system_derivative()

```
system_derivative_t * compute_system_derivative (
          const fish_system_t * system,
          const derivative_computation_opts_t dc_opts )
```

Computes the derivative for a fish system.

Parameters

in	*system*	The system to compute the derivative of.
in	*dc_opts*	The derivative computation options.

derivative_average()

```
system_derivative_t * derivative_average (
          const derivative_weight_t terms[],
          const size_t N,
          const size_t len )
```

Computes the weighted average of a derivative.

Parameters

in	*terms*	The terms to compute on.
in	*N*	The size of the outupt derivative.

Returns

The weighted average derivative.

derivative_destroy()

```
void derivative_destroy (
          system_derivative_t ** derivative_ptr )
```

De-allocates a derivative object.

Parameters

out	*derivative_ptr*	The pointer to the derivative object.

derivative_has_nan()

```
bool derivative_has_nan (
          const system_derivative_t * derivative )
```

Determines whether a system derivative contains a NaN or Inf value (for debugging).

Parameters

in	*derivative*	The system derivative.

Returns

Whether the system derivative contains a non-finite value.

derivative_print()

```
void derivative_print (
          const system_derivative_t * derivative )
```

Prints a derivative (for debugging).

Parameters

in	*derivative*	The system derivative.

6.5 include/finds/error.h File Reference

Basic error handling module.

```
#include <errno.h>
```

Macros

- #define WRAP_HDF5_CHECK(errcode, jmpto, result)
- #define WRAP_CHECK(errcode, jmpto, prev_errcode)
- #define RAISE_ERROR(errtype, errmsg)
- #define RAISE_ERROR_ERRNO(errtype, errmsg)
- #define GOTO_IF_NULL(errcode, jmpto, ptr)
- #define **WARN**(msg) fprintf(stderr, "WARNING: %s\n", msg)

Enumerations

- enum error_e {
  ERR_DATASTREAM_OPEN , **ERR_DATASTREAM_CLOSED** , **ERR_DATASTREAM_WRITE** , **ERR_INVALID_ARG** ,
  ERR_ALLOC , **ERR_STATE_CHANGE** , **ERR_INVALID_STATE** , **ERR_FAILURE** ,
  ERR_OK , **ERR_TYPE_COUNT** }

  Enum for error types.

6.5.1 Detailed Description

Basic error handling module.

This file contains some simple macros for producing and handling errors.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.5.2 Macro Definition Documentation

GOTO_IF_NULL

```
#define GOTO_IF_NULL(
            errcode,
            jmpto,
            ptr )
```

Value:

```
    { if (ptr == NULL) {                        \
            errcode = ERR_FAILURE;              \
            goto jmpto;                         \
        }}
```

RAISE_ERROR

```
#define RAISE_ERROR(
            errtype,
            errmsg )
```

Value:

```
            ({                      \
            fprintf(stderr, "%s:%d %s: %s\n",               \
                __FILE__, __LINE__, __func__, (errmsg));    \
            (errtype);                                      \
                })
```

RAISE_ERROR_ERRNO

```
#define RAISE_ERROR_ERRNO(
            errtype,
            errmsg )
```

Value:

```
({          \
fprintf(stderr, "%s:%d %s: %s (%s)\n",      \
    __FILE__, __LINE__,                     \
    __func__, (errmsg), strerror(errno));   \
(errtype);                                  \
    })
```

WRAP_CHECK

```
#define WRAP_CHECK(
            errcode,
            jmpto,
            prev_errcode )
```

Value:

```
    if (prev_errcode != ERR_OK) {                   \
        errcode = prev_errcode;                     \
        goto jmpto;                                 \
    }
```

Wrapping function for native errors

WRAP_HDF5_CHECK

```
#define WRAP_HDF5_CHECK(
            errcode,
            jmpto,
            result )
```

Value:

```
    ({ if (result < 0) {                        \
            errcode = ERR_DATASTREAM_WRITE;     \
            goto jmpto;                         \
        } (result); })
```

Wrapping function for HDF5 errors

6.6 include/finds/evaluation.h File Reference

Derivative timing.

Functions

- double time_derivative_computation (const fish_system_t *system, const derivative_computation_opts_t dc_opts)

  Times the computation of a single derivative.

6.6.1 Detailed Description

Derivative timing.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.6.2 Function Documentation

time_derivative_computation()

```
double time_derivative_computation (
          const fish_system_t * system,
          const derivative_computation_opts_t dc_opts )
```

Times the computation of a single derivative.

Parameters

in	*system*	The system to compute the derivative of.
in	*dc_opts*	The derivative computation options.

Returns

The runtime required to compute the derivative.

6.7 include/finds/finds.h File Reference

```
#include "barneshut.h"
#include "constants.h"
```

```
#include "datastream.h"
#include "derivative.h"
#include "error.h"
#include "evaluation.h"
#include "finds.h"
#include "integration.h"
#include "interaction.h"
#include "simulation.h"
#include "system.h"
#include "util.h"
#include "vector.h"
```

6.8 include/finds/integration.h File Reference

Implementation of various adaptive and non-adaptive Runge-Kutta integrators.

```
#include <stdbool.h>
#include "system.h"
#include "derivative.h"
```

Classes

- struct integration_opts_t

  Struct for integration options.

Macros

- #define DECLARE_INTEGRATOR(name)

  Simple macro for declaring integrators (as all use the same args)

Typedefs

- typedef fish_system_t ∗(∗ **integration_step_fn**) (const fish_system_t ∗state, const double time_step, const derivative_computation_opts_t dc_opts, double ∗error)

Enumerations

- enum integration_method_e { **EULER** , **RUNGE_KUTTA_2** , **RUNGE_KUTTA_4** , **RUNGE_KUTTA_5** , **RUNGE_KUTTA_6** , **RUNGE_KUTTA_32** , **RUNGE_KUTTA_54** , **INTEGRATION_METHODS_COUNT** }

  Enum for integration methods.

Functions

- **DECLARE_INTEGRATOR** (step_euler)
- **DECLARE_INTEGRATOR** (step_rk2)
- **DECLARE_INTEGRATOR** (step_rk4)
- **DECLARE_INTEGRATOR** (step_rk5)
- **DECLARE_INTEGRATOR** (step_rk6)
- **DECLARE_INTEGRATOR** (step_rk32)
- **DECLARE_INTEGRATOR** (step_rk54)

Variables

- const named_enum_t INTEGRATION_METHODS_TABLE [INTEGRATION_METHODS↩ _COUNT]

6.8.1 Detailed Description

Implementation of various adaptive and non-adaptive Runge-Kutta integrators.

This file contains the core routine for computing a singular integral time-step of the fish system (including optional error estimation).

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.8.2 Macro Definition Documentation

DECLARE_INTEGRATOR

```
#define DECLARE_INTEGRATOR(
            name )
```

Value:

```
fish_system_t *name( \
    const fish_system_t *state, \
    const double time_step, \
    const derivative_computation_opts_t dc_opts, \
    double *error \
)
```

Simple macro for declaring integrators (as all use the same args)

6.8.3 Variable Documentation

INTEGRATION_METHODS_TABLE

```
const named_enum_t INTEGRATION_METHODS_TABLE[INTEGRATION_METHODS_COUNT]
[extern]
```

Named lookup table for integration methods

6.9 include/finds/interaction.h File Reference

Compuatation function for calculating the Laplacian interaction between two fish or two features.

```
#include <math.h>
#include "constants.h"
#include "vector.h"
#include "util.h"
#include "system.h"
```

Typedefs

- typedef double_3d_t(* **interaction_fn_t**) (const double_3d_t feat_a_pos, const double_3d_t feat_b_pos, const double regularization_epsilon)

6.9.1 Detailed Description

Compuatation function for calculating the Laplacian interaction between two fish or two features.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.10 include/finds/simulation.h File Reference

Core simulation function.

```
#include <stdbool.h>
#include "integration.h"
#include "error.h"
```

Functions

- error_e perform_simulation (const fish_system_t *initial_state, const derivative_computation_opts_t dc_opts, const integration_opts_t int_opts, char *output_filename, char *output_folder_name, const size_t output_filename_size, const size_t output_folder_size, const bool print_file_output)

  Performs a simulation using the given initial conditions.

6.10.1 Detailed Description

Core simulation function.

This file contains the routine for performing a simulation over a specified integration time domain.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.10.2 Function Documentation

perform_simulation()

```
error_e perform_simulation (
          const fish_system_t * initial_state,
          const derivative_computation_opts_t dc_opts,
          const integration_opts_t int_opts,
          char * output_filename,
          char * output_folder_name,
          const size_t output_filename_size,
          const size_t output_folder_size,
          const bool print_file_output )
```

Performs a simulation using the given initial conditions.

Runs the simulation based on the initial conditions and calculation parameters. Outputs to a file and produces both the filepath and the parent folder filepath.

Parameters

in	*initial_state*	The initial state of the system.
in	*dc_opts*	The derivative computation options.
in	*int_opts*	The integral computation options.
out	*output_filename*	The output filename buffer for the dataset.
out	*output_folder_name*	The output folder buffer.
in	*output_filename_size*	The size of the output filename buffer.

Returns

The error code for the simulation.

6.11 include/finds/system.h File Reference

Core particle system data structures and operations.

```
#include <string.h>
#include <stdbool.h>
#include <stdint.h>
#include <stddef.h>
#include "vector.h"
#include "util.h"
```

Classes

- struct distribution_options_t

  Distribution generation options struct.
- struct orientation_options_t

  Orientation generation options struct.
- struct constant_options_t

  Constant generation options struct.
- struct swimmer_t

  Structure representing an individual swimmer.
- struct fish_system_t

  Struct for a system of swimmers (i.e., a vector of swimmers).
- struct swimmer_features_t

  Structure of feature (source and sink) positions.
- struct feature_positions_t

  Vector of feature positions.

Enumerations

- enum distribution_type_e {
  DISTRIBUTION_CUBE , **DISTRIBUTION_SPHERE** , **DISTRIBUTION_BALL** , **DISTRIBUTION_RANDOM** ,
  DISTRIBUTION_TYPE_COUNT }

  Enum for distribution generation types.
- enum orientation_type_e {
  ORIENTATION_RANDOM , **ORIENTATION_RADIAL_INWARD** , **ORIENTATION_RADIAL_OUTWARD** , **ORIENTATION_SWIRL_INWARD** ,
  ORIENTATION_SWIRL_OUTWARD , **ORIENTATION_SADDLE** , **ORIENTATION_ALIGNED** , **ORIENTATION_TYPE_COUNT** }

  Enum for orientation generation types.

Functions

- fish_system_t * fish_system_generate (const distribution_options_t dist_opts, const orientation_options_t ori_opts, const constant_options_t const_opts, const bool print_↩ debug)

  Generates a system of fish (positions and orientations).
- fish_system_t * fish_system_combine (const fish_system_t *a, const fish_system_t *b)

  Copies two systems to a greater system containing both.
- fish_system_t * fish_system_combine_destroy (fish_system_t *a, fish_system_t *b)

  Combines two fish systems and then destroys the previous systems.
- fish_system_t * fish_system_difference (const fish_system_t *a, const fish_system_t *b)

  Computes a fish-system representation of the difference.
- double fish_system_norm (const fish_system_t *system)

  Computes the norm of a fish system.
- void fish_system_normalize_orientation (fish_system_t *system)

  Normalizes the orientations for the fish system.
- bool fish_system_has_nan (const fish_system_t *system)

  Determines whether a system constains a NaN value (for debugging).
- fish_system_t * fish_system_generate_random (const size_t N, const bool print_debug)

  Generates a random system with predefined distribution and orientation options (both random) as a function of N, the number of swimmers in the system.
- fish_system_t * fish_system_copy (const fish_system_t *system)

  Produces a fresh copy of a fish system.
- fish_system_t * fish_system_allocate (const size_t N)

  Allocates all of the arrays for a fish system to size N.
- void fish_system_print (const fish_system_t *system)

  Prints the system to stdout (for debugging).
- void fish_system_destroy (fish_system_t **system)

  Destroys the fish system (deallocates all pointers then redirects to NULL).
- void fish_system_translate (fish_system_t *system, const double delta_x, const double delta_y, const double delta_z)

  Translates all of the positions in a fish system.
- void fish_system_rotate (fish_system_t *system, const double roll, const double pitch, const double yaw)

  Rotates all of the positions in a fish system.
- void calculate_swimmer_features (const double_3d_t position, const double_3d_t orientation, const double length, double_3d_t *source_position_ptr, double_3d_t *sink_↩ position_ptr)

  Computes the feature positions for a singular swimmer.
- feature_positions_t * calculate_feature_positions (const fish_system_t *system)

  Calculates the feature positions for a fish system.
- void feature_positions_print (const feature_positions_t *feat_pos)

Prints the feature positions to stdout.

- feature_positions_t * feature_positions_allocate (const size_t N)

  Allocates the feature positions array.

- void feature_positions_destroy (feature_positions_t **feat_pos)

  De-allocates the feature positions structure.

Variables

- const named_enum_t **DISTRIBUTION_TYPE_TABLE** [DISTRIBUTION_TYPE_COUNT]

  Enum naming table for distribution generation types.

- const named_enum_t **ORIENTATION_TYPE_TABLE** [ORIENTATION_TYPE_COUNT]

  Enum naming table for orientation generation types.

6.11.1 Detailed Description

Core particle system data structures and operations.

This file contains routines for creating, initializing, updating, and destroying static systems of fish-particles.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.11.2 Function Documentation

calculate_feature_positions()

```
feature_positions_t * calculate_feature_positions (
          const fish_system_t * system )
```

Calculates the feature positions for a fish system.

Feature positions are calculated from the position and orientation as

source position = position + length / 2 * orientation sink position = position - length / 2 * orientation

Parameters

in	*system*	The system to compute on.

Returns

The positions of the sources and sinks.

calculate_swimmer_features()

```
void calculate_swimmer_features (
          const double_3d_t position,
          const double_3d_t orientation,
          const double length,
          double_3d_t * source_position_ptr,
          double_3d_t * sink_position_ptr )
```

Computes the feature positions for a singular swimmer.

Parameters

in	*position*	The position of the swimmer.
in	*orientation*	The orientation of the swimmer.
in	*length*	The length of the swimmer.
out	*source_position_ptr*	The output vector for the source position.
out	*sink_position_ptr*	The output vector for the sink position.

feature_positions_allocate()

```
feature_positions_t * feature_positions_allocate (
          const size_t N )
```

Allocates the feature positions array.

Parameters

in	*The*	size of the system.

feature_positions_destroy()

```
void feature_positions_destroy (
            feature_positions_t ** feat_pos_ptr )
```

De-allocates the feature positions structure.

Parameters

out	*feat_pos_ptr*	The feature positions pointer.

feature_positions_print()

```
void feature_positions_print (
            const feature_positions_t * feat_pos )
```

Prints the feature positions to stdout.

Parameters

in	*feat_pos*	The feature positions.

fish_system_allocate()

```
fish_system_t * fish_system_allocate (
            const size_t N )
```

Allocates all of the arrays for a fish system to size N.

Parameters

in	*N*	The size of the fish system.

Returns

The new allocated system.

fish_system_combine()

```
fish_system_t * fish_system_combine (
```

```
        const fish_system_t * a,
        const fish_system_t * b )
```

Copies two systems to a greater system containing both.

Parameters

in	*a*	The first fish system.
in	*b*	The second fish system.

Returns

The combined system.

fish_system_combine_destroy()

```
fish_system_t * fish_system_combine_destroy (
        fish_system_t * a,
        fish_system_t * b )
```

Combines two fish systems and then destroys the previous systems.

Parameters

in	*a*	The first fish system.
in	*b*	The second fish system.

Returns

The combined system.

fish_system_copy()

```
fish_system_t * fish_system_copy (
        const fish_system_t * system )
```

Produces a fresh copy of a fish system.

Parameters

in	*system*	The fish system to copy.

Returns

The copied system.

fish_system_destroy()

```
void fish_system_destroy (
          fish_system_t ** system_ptr )
```

Destroys the fish system (deallocates all pointers then redirects to NULL).

Parameters

out	*system_ptr*	The system pointer to destroy.

fish_system_difference()

```
fish_system_t * fish_system_difference (
          const fish_system_t * a,
          const fish_system_t * b )
```

Computes a fish-system representation of the difference.

Parameters

in	*a*	The first system.
in	*b*	The second system.

Returns

The difference.

fish_system_generate()

```
fish_system_t * fish_system_generate (
          const distribution_options_t dist_opts,
          const orientation_options_t ori_opts,
          const constant_options_t const_opts,
          const bool print_debug )
```

Generates a system of fish (positions and orientations).

Parameters

in	*dist_opts*	Distribution options.
in	*ori_opts*	Orientation options.
in	*const_opts*	Constant generation options.
in	*print_debug*	Whether to print out that this system was generated.

Returns

The system.

fish_system_generate_random()

```
fish_system_t * fish_system_generate_random (
          const size_t N,
          const bool print_debug )
```

Generates a random system with predefined distribution and orientation options (both random) as a function of N, the number of swimmers in the system.

Parameters

in	*N*	The number of swimmers to generate.

fish_system_has_nan()

```
bool fish_system_has_nan (
          const fish_system_t * system )
```

Determines whether a system constains a NaN value (for debugging).

Parameters

in	*system*	The system.

Returns

Whether the system contains NaN.

fish_system_norm()

```
double fish_system_norm (
          const fish_system_t * system )
```

Computes the norm of a fish system.

The norm of a fish system is defined as the square root of the sums of the squared-norms of the positions and orientations for all of the fish in the system,

$$|\mathbf{X}|^2 = \sum_{i=1}^{N} \left(|\mathbf{x}_{c,i}|^2 + |\mathbf{n}_i|^2 \right).$$

Parameters

in	*system*	The system to compute the norm of.

Returns

The norm of the system.

fish_system_normalize_orientation()

```
void fish_system_normalize_orientation (
          fish_system_t * system )
```

Normalizes the orientations for the fish system.

Parameters

out	*system*	The fish system to normalize.

fish_system_print()

```
void fish_system_print (
          const fish_system_t * system )
```

Prints the system to stdout (for debugging).

Parameters

in	*system*	The fish system to print.

fish_system_rotate()

```
void fish_system_rotate (
          fish_system_t * system,
          const double roll,
          const double pitch,
          const double yaw )
```

Rotates all of the positions in a fish system.

Parameters

out	*system*	The system to rotate.
in	*roll*	The x-axis rotation angle.
in	*pitch*	The y-axis rotation angle.
in	*yaw*	The z-axis rotation angle.

fish_system_translate()

```
void fish_system_translate (
          fish_system_t * system,
          const double delta_x,
          const double delta_y,
          const double delta_z )
```

Translates all of the positions in a fish system.

Parameters

out	*system*	The system to translate.
in	*delta_x*	The x-displacement.
in	*delta_y*	The y-displacement.
in	*delta_z*	The z-displacement.

6.12 include/finds/util.h File Reference

Utility functions.

```
#include <string.h>
#include <stdlib.h>
#include <time.h>
```

Classes

- struct named_enum_t

  Enum to name string mapping struct.

Macros

- #define **MAX**(a, b) ((a) > (b) ? (a) : (b))

  Returns the max of a and b.
- #define **MIN**(a, b) ((a) < (b) ? (a) : (b))

  Returns the min of a and b.
- #define CLAMP(val, min, max) (MAX((min), MIN((val), (max))))

  Clamps a value between [min,max].
- #define ABOUT_ZERO 1E-20
- #define **ABS**(a) ((a) > 0 ? (a) : -(a))

  Absolute value macro.
- #define **IS_CLOSE**(a, b) (ABS((a) - (b)) < ABOUT_ZERO)

  Comparison macro for comparing floating-point doubles.
- #define **UNUSED** __attribute__((unused))

  Shortened alias for unused parameters.
- #define NOT_IMPLEMENTED()

  Macro for flagging functions that have not been implemented yet.

6.12.1 Detailed Description

Utility functions.

This file contains routines misc. routines for various computations.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.12.2 Macro Definition Documentation

ABOUT_ZERO

```
#define ABOUT_ZERO 1E-20
```

About zero (with regard to the precision of this system).

CLAMP

```
#define CLAMP(
            val,
            min,
            max ) (MAX((min), MIN((val), (max))))
```

Clamps a value between [min,max].

Parameters

in	*val*	The value to clamp.
in	*min*	The minimum value.
in	*max*	The maximum value.

Returns

If min < val < min, then val. If val < min, then min. If val > max, then max.

NOT_IMPLEMENTED

```
#define NOT_IMPLEMENTED( )
```

Value:

```
do { \
    fprintf(stderr, "Function not implemented: %s\n", __func__); \
    abort(); \
} while (0)
```

Macro for flagging functions that have not been implemented yet.

6.13 include/finds/vector.h File Reference

Basic vector and matrix computations.

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <stdbool.h>
#include "util.h"
```

Classes

- union double_3d_t

  3-dimensional cartesian vector implementation
- struct mat3_t

  3x3 matrix implementation

Macros

- #define **D3**(x, y, z) ((double_3d_t) {x,y,z})

  Shorthand for making D3 vectors.
- #define **D3_FILL**(a) D3(a,a,a)

  Fills a D3 with one value on each dimension.
- #define **D3_ONE** D3_FILL(1)

  A D3 of only 1s (1,1,1).
- #define **D3_ZERO** D3_FILL(0)

  A D3 of only 0s (0,0,0)
- #define MAT3(m11, m12, m13, m21, m22, m23, m31, m32, m33)

  Shorthand for creating 3x3 matrices.

6.13.1 Detailed Description

Basic vector and matrix computations.

This file contains routines for operating on 3-dimensional vectors and 3-by-3 matrices. It's focused mostly on speed, and as such, is incredibly small.

Author

Mufaro Machaya mufaro2@student.ubc.ca

6.13.2 Macro Definition Documentation

MAT3

```
#define MAT3(
            m11,
            m12,
            m13,
            m21,
            m22,
            m23,
            m31,
            m32,
            m33 )
```

Value:

```
    ((mat3_t){ .data = {                            \
            {m11, m12, m13},                        \
            {m21, m22, m23},                        \
            {m31, m32, m33}                         \
        }})
```

Shorthand for creating 3x3 matrices.

6.14 programs/benchmark.c File Reference

Produces a benchmarking-plot for the entirety of FINDS.

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <gsl/gsl_fit.h>
#include <gnuplot_i/gnuplot_i.h>
#include <finds/system.h>
#include <finds/datastream.h>
#include <finds/constants.h>
#include <finds/derivative.h>
#include <finds/util.h>
#include <finds/evaluation.h>
```

Macros

- #define **PDF**
- #define **OUTPUT_MODE** "pdf"
- #define **FIGURE_WIDTH** 3.5
- #define **FIGURE_HEIGHT** 2.9
- #define **FONT** "Nimbus Roman,11"
- #define **SERIES_LABEL_SIZE** 200
- #define **SHOW_LEGEND_EXTENDED_DETAILS**
- #define **BRUTE_MAX_LOG_N** 4
- #define **BH_MAX_LOG_N** 7
- #define **FMM_MAX_LOG_N** 8
- #define **MIN_LOG_N** 1
- #define **MAX_LOG_N** 6
- #define **N_METHODS** 9

Functions

- int **main** (void)

6.14.1 Detailed Description

Produces a benchmarking-plot for the entirety of FINDS.

Times the computation of a derivative for variable-N systems for each type of interaction computation method (brute force, Barnes-Hut at increasing theta, and the Fast-Multipole-Method at increasing order).

Author

Mufaro Machaya

License: MIT

6.15 programs/benchmark_derivative.c File Reference

Times the computation of a single derivative.

```
#include <argp.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <finds/system.h>
#include <finds/derivative.h>
#include <finds/util.h>
```

Classes

- struct benchmark_options_t

Macros

- #define **DEFAULT_SIZE** 100
- #define **DEFAULT_METHOD** BRUTE_FORCE
- #define **DEFAULT_APPROX_THRESHOLD** 0.0
- #define **DEFAULT_PRECISION** 1E-6

Functions

- int **main** (int argc, char ∗argv[ ])

Variables

- const char ∗ **argp_program_version** = "FINDS derivative timer v1.0"
- const char ∗ argp_program_bug_address

6.15.1 Detailed Description

Times the computation of a single derivative.

Author

Mufaro Machaya

License: MIT

6.15.2 Variable Documentation

argp_program_bug_address

```
const char* argp_program_bug_address
```

Initial value:

```
=
    "Mufaro J. Machaya <mufaro2@student.ubc.ca>"
```

6.16 programs/linker_test.c File Reference

Just a linker test that includes all of the libraries used in FINDS.

```
#include <argp.h>
#include <math.h>
#include <time.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <error.h>
#include <CL/cl.h>
#include <gsl/gsl_fit.h>
#include <gnuplot_i/gnuplot_i.h>
#include <progressbar/progressbar.h>
#include <tomlc17.h>
#include <hdf5.h>
#include <lfmm3d_c.h>
#include <finds/constants.h>
#include <finds/datastream.h>
#include <finds/derivative.h>
#include <finds/evaluation.h>
#include <finds/integration.h>
#include <finds/simulation.h>
#include <finds/system.h>
#include <finds/util.h>
#include <finds/vector.h>
```

Functions

- int **main** (void)

6.16.1 Detailed Description

Just a linker test that includes all of the libraries used in FINDS.

Author

Mufaro Machaya

License: MIT

6.17 programs/simulate.c File Reference

The core entrypoint for performing a simulation.

```
#include <argp.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <tomlc17.h>
#include <error.h>
#include <finds/system.h>
#include <finds/derivative.h>
#include <finds/simulation.h>
#include <finds/integration.h>
#include <finds/util.h>
#include <finds/constants.h>
```

Classes

- struct cmdline_args_t
- struct finds_opts_t

Macros

- #define **DEFAULT_SIMULATION_FILE** "default-sim-config.toml"
- #define **BUFFER_SIZE** 2048

Functions

- int **main** (int argc, char ∗argv[ ])

Variables

- const char ∗ **argp_program_version** = "FINDS simulation v1.0"
- const char ∗ argp_program_bug_address

6.17.1 Detailed Description

The core entrypoint for performing a simulation.

Performs a simulation based on a user-supplied configuration file, then produces an output trajectory dataset in HDF5 format alongside a copy of the user-supplied configuration TOML.

Author

Mufaro Machaya

License: MIT

6.17.2 Variable Documentation

argp_program_bug_address

```
const char* argp_program_bug_address
```

Initial value:

```
=
    "Mufaro J. Machaya <mufaro2@student.ubc.ca>"
```

6.18 programs/validation.c File Reference

Produces the validation plots for FINDS.

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>
#include <float.h>
#include <gnuplot_i/gnuplot_i.h>
#include <finds/constants.h>
#include <finds/system.h>
#include <finds/simulation.h>
#include <finds/error.h>
#include <finds/datastream.h>
```

Classes

- struct system_trajectory_t

  Swimmer trajectory object for simplifying the coplanar cases.

Macros

- #define **VALIDATION_OUTPUT_FOLDER** ROOT_OUTPUT_PATH "/validation"
- #define **TEMP_OUTPUT_FILE** VALIDATION_OUTPUT_FOLDER "/validation-circle-output.dat"
- #define **LENGTH** 1
- #define **VOLUMETRIC_FLOW_RATE** (4 ∗ M_PI)
- #define **SP_SPEED** (VOLUMETRIC_FLOW_RATE / (4 ∗ M_PI ∗ LENGTH ∗ LENGTH))
- #define **BUFFER_SIZE** 1024
- #define **DOUBLE_BUFFER_SIZE** (2 ∗ BUFFER_SIZE)
- #define **PDF**
- #define **OUTPUT_MODE** "pdf"
- #define **LINEWIDTH** 1
- #define **FIGURE_WIDTH** 3.5
- #define **FIGURE_HEIGHT** 4
- #define **FONT** "Nimbus Roman,11"
- #define **N_DC_METHODS** 5
- #define __WRAP_ERR(errcode, fn)

Functions

- fish_system_t ∗ **comparison_plot_load_initial_conditions** (FILE ∗restrict fptr, const size_t N)
- int **main** (void)

6.18.1 Detailed Description

Produces the validation plots for FINDS.

Reproduces figures 8 and 16 of Mabrouk and Floryan 2025 and 2024 respectively using FINDS and GNUPlot.

Author

Mufaro Machaya

License: MIT

6.18.2 Macro Definition Documentation

__WRAP_ERR

```
#define __WRAP_ERR(
            errcode,
            fn )
```

Value:

```
    errcode = fn;                               \
    if (errcode != ERR_OK)                      \
        return EXIT_FAILURE;
```

6.19 src/barneshut.c File Reference

External velocity contribution calculation through Barnes-Hut.

```
#include <assert.h>
#include <stdlib.h>
#include <stdbool.h>
#include <finds/barneshut.h>
#include <finds/system.h>
#include <finds/vector.h>
#include <finds/interaction.h>
```

Classes

- struct morton_key_t

  This maps morton codes to node indices in a linear octree.
- struct linear_octree_builder_t

  A variable-sized wrapper for building linear octrees.

Macros

- #define MORTON_BITS 21

  The number of bits for storing morton codes (and subsequently, the maximum number of levels for the octree).
- #define **LINEAR_OCTREE_REALLOC**(arrptr, cap) arrptr = realloc((arrptr), (cap) ∗ sizeof ∗(arrptr))

  Simple macro for reallocating the size of an octree (only for internal use when building).
- #define **VECTOR_CLUSTER_APPEND**(sum, vec, w) ((sum) = d3_add((sum), d3_↩ mult((vec), (w))))

  Simple macro for adding vector data to a cluster sum.

Functions

- void linear_octree_print (const linear_octree_t *restrict tree)

  Prints a full octree to STDOUT (for debugging).
- bool linear_octree_check_nan (const linear_octree_t *restrict tree, bool verbose)

  Checks if a linear octree has any NaN values.
- linear_octree_t * linear_octree_build (const fish_system_t *restrict system)

  Builds a linear octree from a fish system.
- void linear_octree_destroy (linear_octree_t **octree_ptr)

  De-allocates a linear octree.
- void linear_octree_compute_vel_contrib (const linear_octree_t *restrict tree, const size_t swimmer_i_index, const double_3d_t swimmer_i_pos, const swimmer_features_t features_i, const double approx_ratio, const bool regularize, const double eps, double_3d_t *restrict external_source, double_3d_t *restrict external_sink)

  Computes the velocity contribution on a swimmer by all of the non-self swimmers in the system.

6.19.1 Detailed Description

External velocity contribution calculation through Barnes-Hut.

This file contains an implementation of the Barnes-Hut algorithm for external velocity contribution using linear octrees as a backend.

Author

Mufaro Machaya mufaro2@student.ubc.ca

6.19.2 Macro Definition Documentation

MORTON_BITS

```
#define MORTON_BITS 21
```

The number of bits for storing morton codes (and subsequently, the maximum number of levels for the octree).

In essence, there are floor(64 bits / 3 axes) = 21 bits available to each dimension (x, y, and z) using 64-bit integers, allowing our tree to have up to 21 levels (for a theoretical maximum of 9.2E18 nodes in the tree).

6.19.3 Function Documentation

linear_octree_build()

```
linear_octree_t * linear_octree_build (
          const fish_system_t *restrict system )
```

Builds a linear octree from a fish system.

Parameters

in	*system*	The fish system to construct from.

Returns

The constructed linear octree.

linear_octree_check_nan()

```
bool linear_octree_check_nan (
          const linear_octree_t *restrict tree,
          bool verbose )
```

Checks if a linear octree has any NaN values.

Parameters

in	*tree*	The octree.
in	*verbose*	Whether to print the exact value which contains NaN.

Returns

Whether or not any NaN values were found inside the tree.

linear_octree_compute_vel_contrib()

```
void linear_octree_compute_vel_contrib (
          const linear_octree_t *restrict tree,
          const size_t swimmer_i_index,
          const double_3d_t swimmer_i_pos,
          const swimmer_features_t features_i,
          const double approx_ratio,
          const bool regularize,
          const double eps,
          double_3d_t *restrict external_source,
          double_3d_t *restrict external_sink )
```

Computes the velocity contribution on a swimmer by all of the non-self swimmers in the system.

Parameters

in	*tree*	The linear octree.
in	*swimmer_i_index*	The swimmer index for swimmer i.
in	*swimmer_i_pos*	The position of swimmer i.
in	*features_i*	The feature positions of swimmer i.
in	*approx_ratio*	The Barnes-Hut approximation ratio.
in	*regularize*	Whether or not to use regularized interaction.
in	*eps*	Regularization Epsilon
out	*external_source*	The output vec. for external source contrib.
out	*external_sink*	The output vec. for external sink contrib.

linear_octree_destroy()

```
void linear_octree_destroy (
          linear_octree_t ** octree_ptr )
```

De-allocates a linear octree.

Parameters

out	*octree_ptr*	The point to the octree to destroy.

linear_octree_print()

```
void linear_octree_print (
          const linear_octree_t *restrict tree )
```

Prints a full octree to STDOUT (for debugging).

Parameters

in	*tree*	The octree to print.

6.20 src/datastream.c File Reference

I/O routines.

```
#include <errno.h>
#include <limits.h>
```

```
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <stdio.h>
#include <hdf5.h>
#include <finds/vector.h>
#include <finds/error.h>
#include <finds/datastream.h>
```

Functions

- error_e mkdir_p (const char *path, const mode_t mode)

  C implementation of "mkdir -p" command.
- error_e datastream_open_file (datastream_t *stream, const char *filename)

  Opens a dataset file for reading.
- error_e **datastream_read_frame** (datastream_t *stream, const size_t frame, fish_system_t **dest_ptr, double *time)
- error_e datastream_create_file (datastream_t *stream, const char *filename, const size_t N)

  Creates the root file for the datastream and marks it as open.
- error_e datastream_close (datastream_t *stream)

  Closes this datastream.
- error_e datastream_write_system (datastream_t *stream, const fish_system_t *system, const double time)

  Writes the state of a single system out to the datastream.

6.20.1 Detailed Description

I/O routines.

This contains the implementation of the datastream object, which is used for creating and writing to simulation data files.

Author

Mufaro Machaya

License: MIT

6.20.2 Function Documentation

datastream_close()

```
error_e datastream_close (
            datastream_t * stream )
```

Closes this datastream.

Parameters

out	*stream*	The stream to close.

Returns

The error code for this process.

datastream_create_file()

```
error_e datastream_create_file (
          datastream_t * stream,
          const char * filename,
          const size_t N )
```

Creates the root file for the datastream and marks it as open.

Parameters

out	*stream*	The datastream to open.
in	*filename*	The output filename for the dataset.
in	*N*	The number of swimmers to track in this dataset.

Returns

The error code for this process.

datastream_open_file()

```
error_e datastream_open_file (
          datastream_t * stream,
          const char * filename )
```

Opens a dataset file for reading.

Parameters

out	*stream*	The datastream to read to.
in	*filename*	The file to open.

Returns

The error code.

datastream_write_system()

```
error_e datastream_write_system (
          datastream_t * stream,
          const fish_system_t * system,
          const double time )
```

Writes the state of a single system out to the datastream.

Parameters

out	*stream*	The datastream to write to.
in	*system*	The system state to write.
in	*time*	The time marker to write.

Returns

The error code for this operation.

mkdir_p()

```
error_e mkdir_p (
          const char * path,
          const mode_t mode )
```

C implementation of "mkdir -p" command.

Note: I'll be honest, I just stole this from generative AI. It works quite okay, though. Another possible implementation could just directly use mkdir -p via a system call (as this code doesn't need to be portable due to running in a virtual machine).

Parameters

in	*path*	The folder path to create.
in	*mode*	The mkdir creation mode.

Returns

The error code for this process.

6.21 src/evaluation.c File Reference

Performance evaluation.

```
#include <time.h>
#include <finds/system.h>
#include <finds/derivative.h>
#include <finds/util.h>
```

Functions

- double time_derivative_computation (const fish_system_t *system, const derivative_computation_opts_t dc_opts)

  Times the computation of a single derivative.

6.21.1 Detailed Description

Performance evaluation.

This file contains the function implementation for timing the computation of a single derivative.

Author

Mufaro J. Machaya

License: MIT

6.21.2 Function Documentation

time_derivative_computation()

```
double time_derivative_computation (
          const fish_system_t * system,
          const derivative_computation_opts_t dc_opts )
```

Times the computation of a single derivative.

Parameters

in	*system*	The system to compute the derivative of.
in	*dc_opts*	The derivative computation options.

Returns

The runtime required to compute the derivative.

6.22 src/integration.c File Reference

Integrator implementations.

```
#include <finds/system.h>
#include <finds/derivative.h>
#include <finds/integration.h>
#include <finds/error.h>
```

Functions

- fish_system_t * step_euler (const fish_system_t *state, const double time_step, const derivative_computation_opts_t dc_opts, double *error)

  Naive Euler-Stepping without Error Estimation.
- fish_system_t * step_rk2 (const fish_system_t *state, const double time_step, const derivative_computation_opts_t dc_opts, double *error)

  Naive 2nd-order Runge-Kutta.
- fish_system_t * step_rk4 (const fish_system_t *state, const double time_step, const derivative_computation_opts_t dc_opts, double *error)

  Naive 4th-order Runge-Kutta.
- fish_system_t * step_rk5 (const fish_system_t *state, const double time_step, const derivative_computation_opts_t dc_opts, double *error)

  Naive 5th-order Runge-Kutta.
- fish_system_t * step_rk6 (const fish_system_t *state, const double time_step, const derivative_computation_opts_t dc_opts, double *error)

  Naive 6th-order Runge-Kutta.
- fish_system_t * step_rk32 (const fish_system_t *state, const double time_step, const derivative_computation_opts_t dc_opts, double *error)

  3rd-order Runge-Kutta (Bogacki-Shampine) with 2nd order error estimation.
- fish_system_t * step_rk54 (const fish_system_t *state, const double time_step, const derivative_computation_opts_t dc_opts, double *error)

  5th-order Runge-Kutta (Dormand-Prince) with 4th order error estimation.

Variables

- const named_enum_t INTEGRATION_METHODS_TABLE [INTEGRATION_METHODS↩ _COUNT]

6.22.1 Detailed Description

Integrator implementations.

This file contains the numerical integration schemes to be used in performing a simulation over some time domain, including forward Euler and embedded Runge-Kutta of variable order.

Author

Mufaro J. Machaya

License: MIT

6.22.2 Function Documentation

step_euler()

```
fish_system_t * step_euler (
          const fish_system_t * state,
          const double time_step,
          const derivative_computation_opts_t dc_opts,
          double * error )
```

Naive Euler-Stepping without Error Estimation.

Parameters

in	*state*	The system state for this time.
in	*time_step*	The time-step to advance to.
out	*error*	The RMS error-norm for this state.

Returns

The advanced state $y(t + \delta t)$.

step_rk2()

```
fish_system_t * step_rk2 (
            const fish_system_t * state,
            const double time_step,
            const derivative_computation_opts_t dc_opts,
            double * error )
```

Naive 2nd-order Runge-Kutta.

Parameters

in	*state*	The system state for this time.
in	*time_step*	The time-step to advance to.
out	*error*	The RMS error-norm for this state.

Returns

The advanced state $y(t + \delta t)$.

step_rk32()

```
fish_system_t * step_rk32 (
            const fish_system_t * state,
            const double time_step,
            const derivative_computation_opts_t dc_opts,
            double * error )
```

3rd-order Runge-Kutta (Bogacki-Shampine) with 2nd order error estimation.

Parameters

in	*state*	The system state for this time.
in	*time_step*	The time-step to advance to.
out	*error*	The RMS error-norm for this state.

Returns

The advanced state y(t + delta t).

step_rk4()

```
fish_system_t * step_rk4 (
            const fish_system_t * state,
            const double time_step,
            const derivative_computation_opts_t dc_opts,
            double * error )
```

Naive 4th-order Runge-Kutta.

Parameters

in	*state*	The system state for this time.
in	*time_step*	The time-step to advance to.
out	*error*	The RMS error-norm for this state.

Returns

The advanced state $y(t + \delta t)$.

step_rk5()

```
fish_system_t * step_rk5 (
            const fish_system_t * state,
            const double time_step,
            const derivative_computation_opts_t dc_opts,
            double * error )
```

Naive 5th-order Runge-Kutta.

Parameters

in	*state*	The system state for this time.
in	*time_step*	The time-step to advance to.
out	*error*	The RMS error-norm for this state.

Returns

The advanced state $y(t + \delta t)$.

step_rk54()

```
fish_system_t * step_rk54 (
            const fish_system_t * state,
            const double time_step,
            const derivative_computation_opts_t dc_opts,
            double * error )
```

5th-order Runge-Kutta (Dormand-Prince) with 4th order error estimation.

Parameters

in	*state*	The system state for this time.
in	*time_step*	The time-step to advance to.
out	*error*	The RMS error-norm for this state.

Returns

The advanced state y(t + delta t).

step_rk6()

```
fish_system_t * step_rk6 (
            const fish_system_t * state,
            const double time_step,
            const derivative_computation_opts_t dc_opts,
            double * error )
```

Naive 6th-order Runge-Kutta.

Parameters

in	*state*	The system state for this time.
in	*time_step*	The time-step to advance to.
out	*error*	The RMS error-norm for this state.

Returns

The advanced state $y(t + \delta t)$.

6.22.3 Variable Documentation

INTEGRATION_METHODS_TABLE

```
const named_enum_t INTEGRATION_METHODS_TABLE[INTEGRATION_METHODS_COUNT]
```

Initial value:

```
= {
    { EULER,          "euler" },
    { RUNGE_KUTTA_2,  "RK2"   },
    { RUNGE_KUTTA_4,  "RK4"   },
    { RUNGE_KUTTA_5,  "RK5"   },
    { RUNGE_KUTTA_6,  "RK6"   },


    { RUNGE_KUTTA_32, "RK32"  },
    { RUNGE_KUTTA_54, "RK54"  }
}
```

Named lookup table for integration methods

6.23 src/simulation.c File Reference

Core simulation routine.

```
#include <math.h>
#include <time.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <progressbar/progressbar.h>
#include <finds/util.h>
#include <finds/integration.h>
#include <finds/simulation.h>
#include <finds/datastream.h>
#include <finds/constants.h>
```

Functions

- error_e perform_simulation (const fish_system_t *initial_state, const derivative_computation_opts_t dc_opts, const integration_opts_t int_opts, char *output_filename, char *output_folder↩ _name, const size_t output_filename_size, const size_t output_folder_size, const bool print_file_output)

  Performs a simulation using the given initial conditions.

6.23.1 Detailed Description

Core simulation routine.

This file contains the leading function for running a fresh simulation and producing the associated simulation datafile.

Author

Mufaro J. Machaya

License: MIT

6.23.2 Function Documentation

perform_simulation()

```
error_e perform_simulation (
          const fish_system_t * initial_state,
          const derivative_computation_opts_t dc_opts,
          const integration_opts_t int_opts,
          char * output_filename,
          char * output_folder_name,
          const size_t output_filename_size,
          const size_t output_folder_size,
          const bool print_file_output )
```

Performs a simulation using the given initial conditions.

Runs the simulation based on the initial conditions and calculation parameters. Outputs to a file and produces both the filepath and the parent folder filepath.

Parameters

in	*initial_state*	The initial state of the system.
in	*dc_opts*	The derivative computation options.
in	*int_opts*	The integral computation options.
out	*output_filename*	The output filename buffer for the dataset.
out	*output_folder_name*	The output folder buffer.
in	*output_filename_size*	The size of the output filename buffer.
in	*otuput_folder_size*	The size of the output folder buffer.
in	*print_file_output*	Whether to print the output filename.

Returns

The error code for the simulation.

6.24 src/system.c File Reference

Core particle system data structures and operations.

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <finds/system.h>
#include <finds/util.h>
```

Functions

- fish_system_t * fish_system_generate (const distribution_options_t dist_opts, const orientation_options_t ori_opts, const constant_options_t const_opts, const bool print_↩ debug)

  Generates a system of fish (positions and orientations).
- fish_system_t * fish_system_combine (const fish_system_t *a, const fish_system_t *b)

  Copies two systems to a greater system containing both.
- double fish_system_norm (const fish_system_t *system)

  Computes the norm of a fish system.
- fish_system_t * fish_system_difference (const fish_system_t *a, const fish_system_t *b)

  Computes a fish-system representation of the difference.
- fish_system_t * fish_system_combine_destroy (fish_system_t *a, fish_system_t *b)

  Combines two fish systems and then destroys the previous systems.
- void fish_system_normalize_orientation (fish_system_t *system)

  Normalizes the orientations for the fish system.
- void fish_system_print (const fish_system_t *system)

  Prints the system to stdout (for debugging).
- fish_system_t * fish_system_copy (const fish_system_t *system)

  Produces a fresh copy of a fish system.
- fish_system_t * fish_system_generate_random (const size_t N, const bool print_debug)

  Generates a random system with predefined distribution and orientation options (both random) as a function of N, the number of swimmers in the system.
- bool fish_system_has_nan (const fish_system_t *system)

  Determines whether a system constains a NaN value (for debugging).
- fish_system_t * fish_system_allocate (const size_t N)

Allocates all of the arrays for a fish system to size N.

- void fish_system_destroy (fish_system_t **system_ptr)

  Destroys the fish system (deallocates all pointers then redirects to NULL).

- void fish_system_translate (fish_system_t *system, const double delta_x, const double delta_y, const double delta_z)

  Translates all of the positions in a fish system.

- void fish_system_rotate (fish_system_t *system, const double roll, const double pitch, const double yaw)

  Rotates all of the positions in a fish system.

- void calculate_swimmer_features (const double_3d_t position, const double_3d_t orientation, const double length, double_3d_t *source_position_ptr, double_3d_t *sink_↩ position_ptr)

  Computes the feature positions for a singular swimmer.

- feature_positions_t * calculate_feature_positions (const fish_system_t *system)

  Calculates the feature positions for a fish system.

- void feature_positions_print (const feature_positions_t *feat_pos)

  Prints the feature positions to stdout.

- feature_positions_t * feature_positions_allocate (const size_t N)

  Allocates the feature positions array.

- void feature_positions_destroy (feature_positions_t **feat_pos_ptr)

  De-allocates the feature positions structure.

Variables

- const named_enum_t DISTRIBUTION_TYPE_TABLE [DISTRIBUTION_TYPE_COUNT]

  Enum naming table for distribution generation types.

- const named_enum_t ORIENTATION_TYPE_TABLE [ORIENTATION_TYPE_COUNT]

  Enum naming table for orientation generation types.

6.24.1 Detailed Description

Core particle system data structures and operations.

This file contains routines for creating, initializing, updating, and destroying static systems of fish-particles.

Author

Mufaro J. Machaya

License: MIT

6.24.2 Function Documentation

calculate_feature_positions()

```
feature_positions_t * calculate_feature_positions (
          const fish_system_t * system )
```

Calculates the feature positions for a fish system.

Feature positions are calculated from the position and orientation as

source position = position + length / 2 ∗ orientation sink position = position - length / 2 ∗ orientation

Parameters

in	*system*	The system to compute on.

Returns

The positions of the sources and sinks.

calculate_swimmer_features()

```
void calculate_swimmer_features (
          const double_3d_t position,
          const double_3d_t orientation,
          const double length,
          double_3d_t * source_position_ptr,
          double_3d_t * sink_position_ptr )
```

Computes the feature positions for a singular swimmer.

Parameters

in	*position*	The position of the swimmer.
in	*orientation*	The orientation of the swimmer.
in	*length*	The length of the swimmer.
out	*source_position_ptr*	The output vector for the source position.
out	*sink_position_ptr*	The output vector for the sink position.

feature_positions_allocate()

```
feature_positions_t * feature_positions_allocate (
          const size_t N )
```

Allocates the feature positions array.

Parameters

in	*The*	size of the system.

feature_positions_destroy()

```
void feature_positions_destroy (
          feature_positions_t ** feat_pos_ptr )
```

De-allocates the feature positions structure.

Parameters

out	*feat_pos_ptr*	The feature positions pointer.

feature_positions_print()

```
void feature_positions_print (
          const feature_positions_t * feat_pos )
```

Prints the feature positions to stdout.

Parameters

in	*feat_pos*	The feature positions.

fish_system_allocate()

```
fish_system_t * fish_system_allocate (
          const size_t N )
```

Allocates all of the arrays for a fish system to size N.

Parameters

in	*N*	The size of the fish system.

Returns

The new allocated system.

fish_system_combine()

```
fish_system_t * fish_system_combine (
          const fish_system_t * a,
          const fish_system_t * b )
```

Copies two systems to a greater system containing both.

Parameters

in	*a*	The first fish system.
in	*b*	The second fish system.

Returns

The combined system.

fish_system_combine_destroy()

```
fish_system_t * fish_system_combine_destroy (
          fish_system_t * a,
          fish_system_t * b )
```

Combines two fish systems and then destroys the previous systems.

Parameters

in	*a*	The first fish system.
in	*b*	The second fish system.

Returns

The combined system.

fish_system_copy()

```
fish_system_t * fish_system_copy (
            const fish_system_t * system )
```

Produces a fresh copy of a fish system.

Parameters

in	*system*	The fish system to copy.

Returns

The copied system.

fish_system_destroy()

```
void fish_system_destroy (
            fish_system_t ** system_ptr )
```

Destroys the fish system (deallocates all pointers then redirects to NULL).

Parameters

out	*system_ptr*	The system pointer to destroy.

fish_system_difference()

```
fish_system_t * fish_system_difference (
            const fish_system_t * a,
            const fish_system_t * b )
```

Computes a fish-system representation of the difference.

Parameters

in	*a*	The first system.
in	*b*	The second system.

Returns

The difference.

fish_system_generate()

```
fish_system_t * fish_system_generate (
          const distribution_options_t dist_opts,
          const orientation_options_t ori_opts,
          const constant_options_t const_opts,
          const bool print_debug )
```

Generates a system of fish (positions and orientations).

Parameters

in	*dist_opts*	Distribution options.
in	*ori_opts*	Orientation options.
in	*const_opts*	Constant generation options.
in	*print_debug*	Whether to print out that this system was generated.

Returns

The system.

fish_system_generate_random()

```
fish_system_t * fish_system_generate_random (
          const size_t N,
          const bool print_debug )
```

Generates a random system with predefined distribution and orientation options (both random) as a function of N, the number of swimmers in the system.

Parameters

in	*N*	The number of swimmers to generate.

fish_system_has_nan()

```
bool fish_system_has_nan (
          const fish_system_t * system )
```

Determines whether a system constains a NaN value (for debugging).

Parameters

in	*system*	The system.

Returns

Whether the system contains NaN.

fish_system_norm()

```
double fish_system_norm (
          const fish_system_t * system )
```

Computes the norm of a fish system.

The norm of a fish system is defined as the square root of the sums of the squared-norms of the positions and orientations for all of the fish in the system,

$$|\mathbf{X}|^2 = \sum_{i=1}^{N} \left(|\mathbf{x}_{c,i}|^2 + |\mathbf{n}_i|^2 \right).$$

Parameters

in	*system*	The system to compute the norm of.

Returns

The norm of the system.

fish_system_normalize_orientation()

```
void fish_system_normalize_orientation (
          fish_system_t * system )
```

Normalizes the orientations for the fish system.

Parameters

out	*system*	The fish system to normalize.

fish_system_print()

```
void fish_system_print (
          const fish_system_t * system )
```

Prints the system to stdout (for debugging).

Parameters

in	*system*	The fish system to print.

fish_system_rotate()

```
void fish_system_rotate (
          fish_system_t * system,
          const double roll,
          const double pitch,
          const double yaw )
```

Rotates all of the positions in a fish system.

Parameters

out	*system*	The system to rotate.
in	*roll*	The x-axis rotation angle.
in	*pitch*	The y-axis rotation angle.
in	*yaw*	The z-axis rotation angle.

fish_system_translate()

```
void fish_system_translate (
           fish_system_t * system,
           const double delta_x,
           const double delta_y,
           const double delta_z )
```

Translates all of the positions in a fish system.

Parameters

out	*system*	The system to translate.
in	*delta_x*	The x-displacement.
in	*delta_y*	The y-displacement.
in	*delta_z*	The z-displacement.

6.24.3 Variable Documentation

DISTRIBUTION_TYPE_TABLE

```
const named_enum_t DISTRIBUTION_TYPE_TABLE[DISTRIBUTION_TYPE_COUNT]
```

Initial value:

```
= {
    { DISTRIBUTION_CUBE,    "cube" },
    { DISTRIBUTION_SPHERE,  "sphere" },
    { DISTRIBUTION_BALL,    "ball" },
    { DISTRIBUTION_RANDOM,  "random" }
}
```

Enum naming table for distribution generation types.

ORIENTATION_TYPE_TABLE

```
const named_enum_t ORIENTATION_TYPE_TABLE[ORIENTATION_TYPE_COUNT]
```

Initial value:

```
= {
    { ORIENTATION_RANDOM,         "random" },
    { ORIENTATION_RADIAL_INWARD,  "radial-inward" },
    { ORIENTATION_RADIAL_OUTWARD, "radial-outward"},
    { ORIENTATION_SWIRL_INWARD,   "swirl-inward" },
    { ORIENTATION_SWIRL_OUTWARD,  "swirl-outward" },
    { ORIENTATION_SADDLE,         "saddle" },
    { ORIENTATION_ALIGNED,        "aligned" },
}
```

Enum naming table for orientation generation types.

6.25 tests/main.c File Reference

Entry-point for running unit tests.

```
#include <stdio.h>
#include <stdlib.h>
#include <check.h>
#include "tests.h"
```

Functions

- int **main** (void)

6.25.1 Detailed Description

Entry-point for running unit tests.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.26 tests/test_datastream.c File Reference

Unit tests for the Datastream I/O module.

```
#include "tests.h"
#include <check.h>
#include <finds/datastream.h>
#include <finds/constants.h>
```

Macros

- #define **SYSTEM_SIZE** 100
- #define **TEST_OUTPUT_FILE** "test_datafile.h5"
- #define **__WRAP**(code) ck_assert(code == ERR_OK)

Functions

- START_TEST (test_read_write_hdf5)

  Unit test for reading/writing datastream HDF5 files.
- END_TEST Suite * **test_suite_datastream** (void)

6.26.1 Detailed Description

Unit tests for the Datastream I/O module.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.26.2 Function Documentation

START_TEST()

```
START_TEST (
          test_read_write_hdf5  )
```

Unit test for reading/writing datastream HDF5 files.

1. The deserialized version of the data should be identical to the data before serialization. (We should be able to fully retrieve the data afterward.)

6.27 tests/test_derivative.c File Reference

Unit tests for the derivative and interaction calculation module.

```
#include <stdlib.h>
#include <check.h>
#include <finds/derivative.h>
#include "tests.h"
```

Functions

- **START_TEST** (test_compute_derivative_brute_force)
- END_TEST **START_TEST** (test_compute_derivative_barnes_hut)
- END_TEST **START_TEST** (test_compute_derivative_fast_multipole_method)
- END_TEST **START_TEST** (test_regularization)
- END_TEST Suite * **test_suite_derivative** (void)

6.27.1 Detailed Description

Unit tests for the derivative and interaction calculation module.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.28 tests/test_integration.c File Reference

Unit tests for the numeric integration module.

```
#include <check.h>
#include <finds/integration.h>
#include "tests.h"
```

Functions

- **START_TEST** (test_step_euler)
- END_TEST **START_TEST** (test_step_rk4)
- END_TEST Suite * **test_suite_integration** (void)

6.28.1 Detailed Description

Unit tests for the numeric integration module.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.29 tests/test_octree.c File Reference

Unit tests for the linear octree generation.

```
#include <check.h>
#include <finds/barneshut.h>
#include "tests.h"
```

Functions

- **START_TEST** (test_octree_generation)
- END_TEST Suite * **test_suite_octree** (void)

6.29.1 Detailed Description

Unit tests for the linear octree generation.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.30 tests/test_simulation.c File Reference

Unit tests for the main simulation module.

```
#include <check.h>
#include <finds/simulation.h>
#include "tests.h"
```

Functions

- **START_TEST** (test_simulation_non_adaptive_rk4)
- END_TEST **START_TEST** (test_simulation_adaptive_rk45)
- END_TEST Suite * **test_suite_simulation** (void)

6.30.1 Detailed Description

Unit tests for the main simulation module.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

6.31 tests/test_system.c File Reference

Unit tests for the fish system generation module.

```
#include <check.h>
#include <finds/system.h>
#include "tests.h"
```

Functions

- **START_TEST** (test_system_random)
- END_TEST Suite * **test_suite_system** (void)

6.31.1 Detailed Description

Unit tests for the fish system generation module.

Author

Mufaro Machaya mufaro2@student.ubc.ca

License: MIT

Index